ITI · INTRODUCTION TO PYTHON TRACK

Python Programming for AI Applications

Eng. Fatma Elraey

Day 2 — Power Tips, List & Dictionary Methods, and Variable Scope

Field	Detail
Instructor	Fatma Elraey
Track	Python for AI Applications
Session	Day 2 of 6
Focus	Python power tips · list & dictionary methods · variable scope (local/global)

Quick Recap — Day 1

Yesterday we covered Python history, syntax, variables & data types, operators, conditions, loops, and functions. Today builds directly on that: sharper syntax idioms, the methods that make lists and dictionaries useful in real code, and how Python resolves variable names across nested functions.

Day 2 Outline

- Python Power Tips (6 idioms every Pythonista uses)
- List and Dictionary Methods
- Python Scope
- Practical Time — practice questions

01 Python Power Tips

Before we go further into methods and scope, here are six small idioms that instantly make your Python read cleaner — the kind of code you'll see in almost every real-world script and notebook.

Tip 1 — The One-Line if (Ternary Expression)

Do a command if this condition is true, else do the other command — all on one line.

```
canFly = True
bird = "Dove" if canFly else "Penguin"
# bird = "Dove"
```

Tip 2 — split() and join()

join() glues a list of strings together with a separator; split() breaks a string apart at a delimiter — they're opposites.

```
":".join(["1", "Ali", "grp"])     # colon is the separator
# '1:Ali:grp'

" ".join("ITI")                   # space is the separator
# 'I T I'

"Sara Mohamed".split(" ")        # space is the delimiter
# ['Sara', 'Mohamed']

"django:flask".split(":")        # colon is the delimiter
# ['django', 'flask']
```

> **AI CONNECTION** *split() is the first step of almost every text-cleaning pipeline — breaking raw text into tokens before it ever reaches a model.*

Tip 3 — Swapping Variables

Python lets you swap two variables in a single line, no temporary variable required.

```
# Traditional way
x = 4
y = 5
temp = x
x = y
y = temp

# Python way
x, y = 4, 5
x, y = y, x
```

Tip 4 — enumerate()

enumerate() gives you both the index and the value while looping — no manual counter needed.

```
languages = ["JavaScript", "Python", "Java"]
for i, l in enumerate(languages):
    print("Element Value: ", l, end=", ")
    print("Element Index: ", i)

# Element Value: JavaScript, Element Index: 0
# Element Value: Python, Element Index: 1
# Element Value: Java, Element Index: 2
```

Tip 5 — is vs ==

== compares values; is compares identity (whether two names point to the exact same object in memory). They usually agree for simple values but not for containers.

```
True == 1        # True   (1 is treated as truthy-equal)
True is 1        # False  (different objects/types)

list1 = [1, 2, 3]
list2 = [1, 2, 3]
list1 == list2   # True   (same contents)
list1 is list2   # False  (two different list objects)
```

Tip 6 — all() and any()

all() checks whether every item in an iterable is truthy; any() checks whether at least one item is truthy.

```
L = [0, 5, 9, 7, 8]
all(L)   # False  (0 is falsy)
any(L)   # True   (at least one truthy value)
```

✓ QUICK CHECK Python Power Tips

Q1. What does "-".join(['2024','01','15']) return?

Answer: *'2024-01-15' — join glues the list items together using '-' as the separator.*

Q2. Is `[1,2] is [1,2]` True or False? Why?

Answer: *False — they're equal in value but are two different list objects in memory, so `is` (identity) fails even though `==` would be True.*

Q3. What does all([]) return for an empty list?

Answer: *True — all() returns True when there are no items to fail the check (vacuously true).*

02 List Methods

Lists come with built-in methods for adding, removing, and rearranging items in place. Let's follow one list through five common operations.

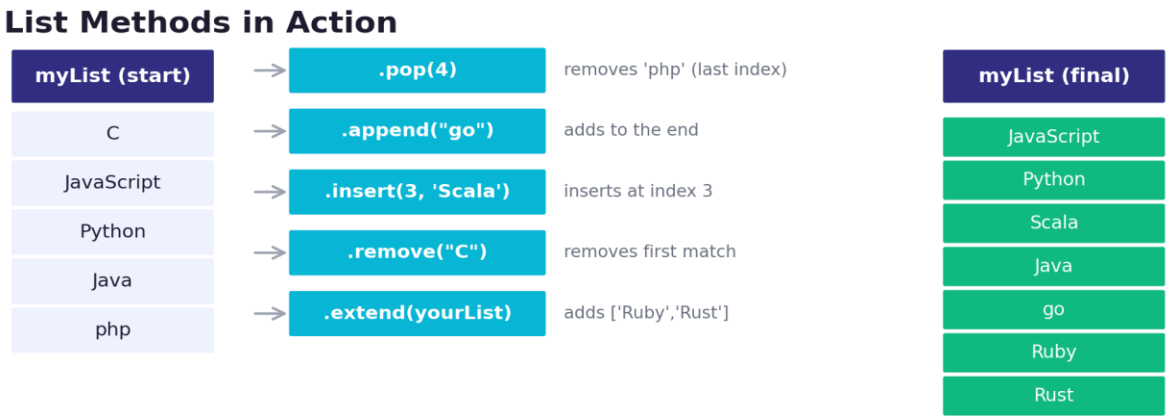

Figure 2.1 — myList starts as 5 items and ends as 7, after five chained method calls.

```
myList = ["C", "JavaScript", "Python", "Java", "php"]

myList.pop(4)              # removes 'php' (by index)
myList.append("go")        # adds 'go' to the end
myList.insert(3, 'Scala')  # inserts 'Scala' at index 3
myList.remove("C")         # removes the first match of 'C'

yourList = ["Ruby", "Rust"]
myList.extend(yourList)    # appends every item from yourList

print(myList)
# ['JavaScript', 'Python', 'Scala', 'Java', 'go', 'Ruby', 'Rust']
```

pop() vs remove() — a common mix-up

pop() takes an index and returns the removed item. remove() takes a value and deletes the first match. Mixing them up is one of the most common list bugs.

```
nums = [10, 20, 30, 40]
nums.pop(1)      # removes the item AT index 1 -> removes 20
nums.remove(30)  # removes the item EQUAL to 30 -> removes 30
```

AI CONNECTION *You'll use append() and extend() constantly when building up batches, feature lists, or prediction results inside a loop.*

03 Dictionary Methods

Dictionaries have their own toolkit for inspecting and merging key–value data — essential for working with structured records like model configs or API responses.

```
infoDict = {'track': 'OS', 'name': 'Ahmed', 'age': 17}

infoDict.keys()      # dict_keys(['track', 'name', 'age'])
'name' in infoDict   # True  -> checks the KEYS, not the values

infoDict.items()
# dict_items([('track', 'OS'), ('name', 'Ahmed'), ('age', 17)])

addInfoDict = {'track': 'SD', 'branch': 'Smart'}
infoDict.update(addInfoDict)
# {'track': 'SD', 'name': 'Ahmed', 'age': 17, 'branch': 'Smart'}
```

Notice update() overwrote 'track' (shared key) but kept 'name' and 'age', and added the new 'branch' key. This is the standard way to merge two dictionaries in place.

AI CONNECTION *update() is exactly how you'd merge a default config dictionary with user-supplied overrides — a pattern you'll see in almost every ML training script.*

✓ QUICK CHECK List & Dictionary Methods

Q1. What's the difference between myList.pop(2) and myList.remove(2)?

Answer: *pop(2) removes the item at index 2; remove(2) removes the first item equal to the value 2. They're easy to confuse.*

Q2. Does 'name' in infoDict check keys or values?

Answer: *Keys — the `in` operator on a dictionary checks membership among its keys by default.*

Q3. If both dictionaries share a key, what does .update() do with it?

Answer: *The value from the dictionary passed into update() overwrites the existing value for that shared key.*

04 Python Scope

Scope determines where a variable name can be seen and used. Python looks names up using the LEGB rule: Local, Enclosing, Global, Built-in — in that order.

Local Shadowing

When a nested function refers to a name, Python searches its own local scope first, then each enclosing function's scope, then the global scope.

```
name = "Ahmed"

def outerFn():
    name = "Ali"
    def innerFn():
        print(name)
    innerFn()

outerFn()
print(name)

# Ali
# Ahmed
```

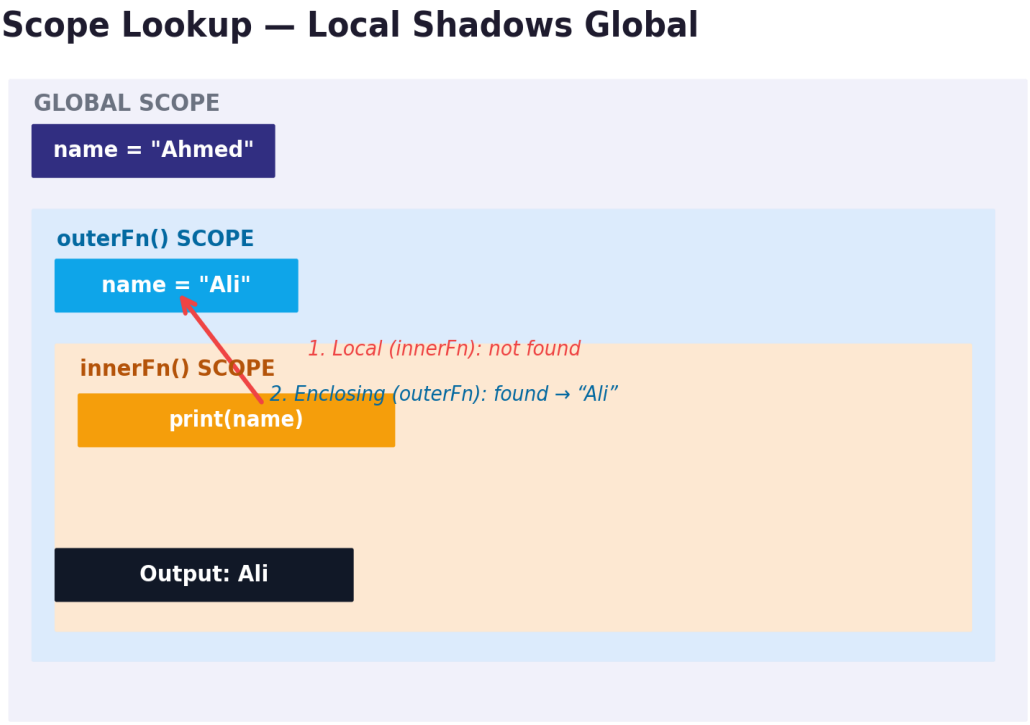

Figure 4.1 — innerFn() has no local 'name', so Python looks outward and finds 'Ali' in outerFn()'s scope.

The final print(name) at the bottom runs in the global scope, where name is still "Ahmed" — outerFn()'s local name = "Ali" never touched the global variable.

The global Keyword

Adding global name inside a function tells Python: don't create a new local variable — modify the global one directly.

```
name = "Ahmed"

def outerFn():
    global name
    name = "Ali"
    def innerFn():
        print(name)
    innerFn()

outerFn()

# Ali
```

Figure 4.2 — with `global name` declared, the assignment updates the global variable instead of shadowing it.

AI CONNECTION *Overusing global state makes training scripts hard to debug — most AI codebases prefer passing values explicitly through function arguments instead.*

✓ **QUICK CHECK Python Scope**

Q1. What does LEGB stand for?

Answer: *Local, Enclosing, Global, Built-in — the order Python searches when looking up a variable name.*

Q2. Without the `global` keyword, does assigning to a name inside a function change the global variable?

Answer: *No — it creates a brand-new local variable that only exists inside that function, leaving the global one untouched.*

Q3. In the shadowing example, why did the final print(name) output 'Ahmed' and not 'Ali'?

Answer: *Because that print() runs in the global scope, and outerFn()'s `name = "Ali"` was local to outerFn() — it never modified the global variable.*

05 Practical Time

Nine practice questions on functions and scope — try to predict the output before checking the marked answer.

Practice Question 1

What is the output of the following code?

```
def outer_fun(a, b):
    def inner_fun(c, d):
        return c + d
    return inner_fun(a, b)

res = outer_fun(5, 10)
print(res)
```

A) 15 ✓

B) Syntax Error

C) (5, 10)

Why: *outer_fun returns whatever inner_fun(a, b) returns, which is c + d = 5 + 10 = 15.*

Practice Question 2

What is the output of the following function call?

```
def fun1(name, age=20):
    print(name, age)

fun1('Emma', 25)
```

A) Emma 25 ✓

B) Emma 20

Why: *25 is passed positionally for age, which overrides the default value of 20.*

Practice Question 3

What is the output of the following display() function call?

```
def display(**kwargs):
    for i in kwargs:
        print(i)

display(emp="Kelly", salary=9000)
```

A) TypeError

B) Kelly 9000

C) ('emp','Kelly') ('salary',9000)

D) emp salary ✓

Why: *Looping over a dictionary with `for i in kwargs` iterates over its KEYS, not the values or (key, value) pairs.*

Practice Question 4

Select which statements are true for Python functions (multiple correct answers).

A) A Python function can return only a single value

B) A function can take an unlimited number of arguments ✓

C) A Python function can return multiple values ✓

D) A Python function doesn't return anything unless you add a return statement

Why: *True: #2 (via *args), #3 (as a tuple), False: #1 — a function can return multiple values packed as a tuple. and #4 (returns None otherwise).*

Practice Question 5

Choose the correct declaration of fun1() so both calls below work.

```
fun1(25, 75, 55)
fun1(10, 20)
```

A) def fun1(**kwargs)

B) No, it is not possible in Python

C) def fun1(args*)

D) def fun1(*data) ✓

Why: *Both calls pass a variable number of POSITIONAL arguments, which requires *args-style syntax — def fun1(*data).*

Practice Question 6

Given def fun1(name, age): print(name, age) — select all correct function calls.

A) fun1("Emma", age=23) ✓

B) fun1(age=23, name="Emma") ✓

C) fun1(name="Emma", 23)

D) fun1(age=23, "Emma")

Why: *Options 1 and 2 are valid. Options 3 and 4 are invalid — a positional argument can never follow a keyword argument.*

Practice Question 7

What is the output of the following display_person() function call?

```
def display_person(*args):
    for i in args:
        print(i)

display_person(name="Emma", age="25")
```

A) TypeError ✓

B) Emma 25

C) name age

Why: **args only collects POSITIONAL arguments. Calling with keyword arguments (name=..., age=...) raises a TypeError.*

Practice Question 8

What is the output of the following function call?

```
def outer_fun(a, b):
    def inner_fun(c, d):
        return c + d
    return inner_fun(a, b)
    return a

result = outer_fun(5, 10)
print(result)
```

A) 5

B) 15 ✓

C) (15, 5)

D) Syntax Error

Why: *The first return statement exits the function immediately — `return a` is unreachable dead code, not a syntax error.*

Practice Question 9

What is the output of the add() function call?

```
def add(a, b):
    return a+5, b+5

result = add(3, 2)
print(result)
```

A) 15

B) 8

C) (8, 7) ✓

D) Syntax Error

Why: *Returning two comma-separated values packs them into a tuple: (3+5, 2+5) = (8, 7).*

Nice work! Next up: Lab 2 — put list/dict methods and scope into practice.